



PROGRAMMING 2B

ASSESSMENT 1: POE



ST10447256
LESEDI PHALATSE

Part 1 – System Planning and Database

Section A – Entity Relationship Diagram (ERD)

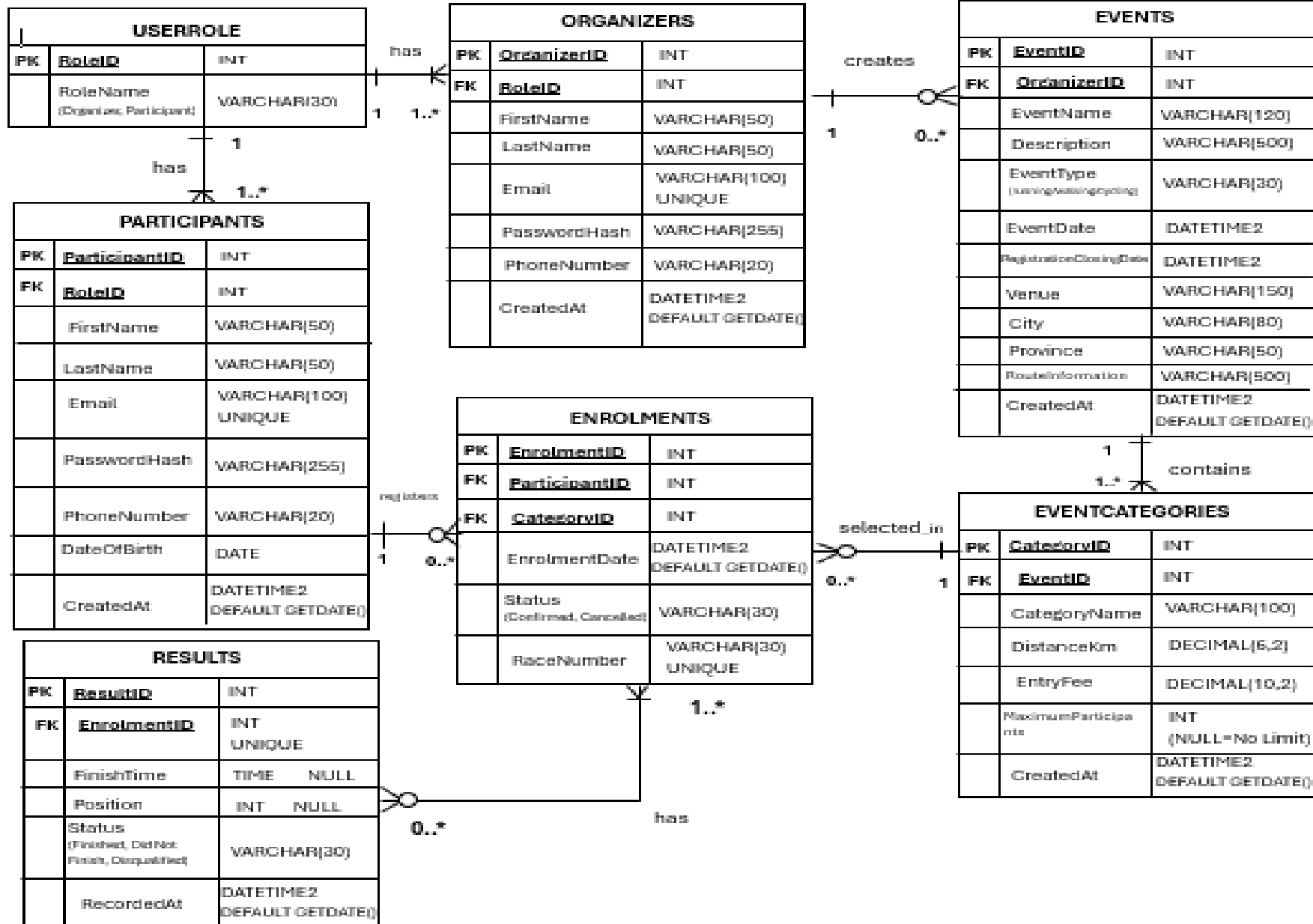
Entity	Purpose
UserRoles	Stores the available system roles: Organiser and Participant
Organizers	Stores the account and profile details of users who organise and manage RaceDay events
Participants	Stores the account and profile details of users who register for and participate in events.
Events	Stores information about road running, walking and cycling events created by organisers.
EventCategories	Stores the different categories available within an event, such as a 5 km, 10 km or 21.1 km race.
Enrolments	Records participants entering an event by selecting a specific event category.
Results	Stores the race results of participants after an event has taken place.

Relationship	Cardinality	Explanation
UserRole - Organizer	1 : Many	One role can be assigned to many organisers, while each organiser is associated with one role.
UserRole – Participants	1 : Many	One role can be assigned to many participants, while each participant is associated with one role.
Organizers – Events	1 : 0..*	One organiser can create zero or many events, while each event must belong to exactly one organiser.

Events - EventCategories	1 : 1..*	One event contains one or more event categories, while each event category belongs to exactly one event.
Participants – Enrolments	1 : 0..*	One participant can have zero or many enrolments, while each enrolment belongs to exactly one participant.
EventCategories - Enrolments	1 : 0..*	One event category can receive zero or many enrolments, while each enrolment must select exactly one event category.
Enrolments - Results	1 : 0..1	One enrolment can have zero or one result, while each result belongs to exactly one enrolment.

The Entity Relationship Diagram (ERD) for the RaceDay system shows the structure of the database and the relationships between its main entities: UserRole, Organizers, Participants, Events, EventCategories, Enrolments, and Results. Relational databases use primary and foreign keys to identify records and create relationships between tables (Microsoft, 2026). In RaceDay, organisers and participants are stored separately because they perform different roles in the system. Organisers are linked to the events they create, while participants are linked to event categories through enrolments. Results are then linked to each participant's enrolment. This structure reduces unnecessary duplication and keeps the database organised and easier to manage (IBM, 2025).

Figure 1 RaceDay Entity Relationship Diagram (ERD) below:



Section B – API Endpoint Plan:

The following API endpoint plan outlines the RESTful services that will be implemented in Part 2 of the RaceDay system. The endpoints define how organisers and participants will interact with authentication, user profiles, events, categories, enrolments, and race results. Each endpoint specifies the HTTP method, route, access role, request data, and expected HTTP responses, including relevant failure cases.

Table 1: Participant API Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register/participant	Registers a new participant account.	None	{ firstName, lastName, email, password, phoneNumber, dateOfBirth }	201 Created – account created; 400 Bad Request – invalid data; 409 Conflict – email exists
POST	/api/auth/login	Authenticates the participant	None	{ email, password }	200 OK – login successful; 400 Bad Request –

		and returns an access token.			missing data; 401 Unauthorized – invalid credentials
GET	/api/participants/profile	Returns the logged-in participant's profile.	Participant	None	200 OK – profile returned; 401 Unauthorized; 403 Forbidden
PUT	/api/participants/profile	Updates the logged-in participant's profile.	Participant	{ firstName, lastName, phoneNumber, dateOfBirth }	200 OK – profile updated; 400 Bad Request; 401 Unauthorized; 403 Forbidden
GET	/api/events	Returns all available RaceDay events for browsing.	None	None	200 OK – events returned

GET	/api/events/{id}	Returns details of a selected event.	None	None	200 OK – event returned; 404 Not Found
GET	/api/events/{eventId}/categories	Returns all categories available for an event.	None	None	200 OK – categories returned; 404 Not Found
GET	/api/categories/{id}	Returns details of a selected event category.	None	None	200 OK – category returned; 404 Not Found
POST	/api/enrolments	Enrols the logged-in participant into a selected event category.	Participant	{ categoryId }	201 Created – enrolled; 400 Bad Request; 403 Forbidden; 404 Not Found; 409 Conflict – already enrolled
GET	/api/enrolments/my	Returns all enrolments	Participant	None	200 OK – enrolments

		belonging to the logged-in participant.			returned; 401 Unauthorized; 403 Forbidden
GET	/api/enrolments/{id}	Returns details of one of the participant's enrolments.	Participant	None	200 OK – enrolment returned; 403 Forbidden – enrolment belongs to another participant; 404 Not Found
DELETE	/api/enrolments/{id}	Cancels an enrolment belonging to the logged-in participant.	Participant	None	204 No Content – cancelled; 403 Forbidden; 404 Not Found
GET	/api/results/my	Returns the participant's	Participant	None	200 OK – results returned; 401

		personal race- result history			Unauthorized; 403 Forbidden
GET	/api/results/{id}	Returns details of a specific result belonging to the participant.	Participant	None	200 OK – result returned; 403 Forbidden; 404 Not Found

Table 2: Organiser API Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register/organizer	Registers a new organiser account.	None	{ firstName, lastName, email, password, phoneNumber }	201 Created – account created; 400 Bad Request – invalid data; 409 Conflict – email exists
POST	/api/auth/login	Authenticates the organiser and returns an access token.	None	{ email, password }	200 OK – login successful; 400 Bad Request; 401 Unauthorized – invalid credentials

GET	/api/organizers/profile	Returns the logged-in organiser's profile.	Organiser	None	200 OK – profile returned; 401 Unauthorized; 403 Forbidden
PUT	/api/organizers/profile	Updates the logged-in organiser's profile.	Organiser	{ firstName, lastName, phoneNumber }	200 OK – profile updated; 400 Bad Request; 401 Unauthorized; 403 Forbidden
GET	/api/events	Returns all RaceDay events.	None	None	200 OK – events returned
GET	/api/events/{id}	Returns details of a specific event.	None	None	200 OK – event returned; 404 Not Found
GET	/api/organizers/events	Returns all events created by the logged-in organiser.	Organiser	None	200 OK – organiser's events returned; 401 Unauthorized; 403 Forbidden
POST	/api/events	Creates a new running, walking or cycling event.	Organiser	{ eventName, description, eventType, eventDate, registrationClosingDate, venue, city, province, routeInformation }	201 Created – event created; 400 Bad Request; 401 Unauthorized; 403 Forbidden
PUT	/api/events/{id}	Updates an event belonging	Organiser	{ eventName, description, eventType, eventDate,	200 OK – updated; 400 Bad Request;

		to the logged-in organiser.		registrationClosingDate, venue, city, province, routeInformation }	403 Forbidden; 404 Not Found
DELETE	/api/events/{id}	Deletes an event belonging to the logged-in organiser.	Organiser	None	204 No Content – deleted; 403 Forbidden; 404 Not Found
GET	/api/events/{eventId}/categories	Returns all categories belonging to an event.	None	None	200 OK – categories returned; 404 Not Found
POST	/api/events/{eventId}/categories	Creates a category for an organiser's event.	Organiser	{ categoryName, distanceKm, entryFee, maximumParticipants }	201 Created – category created; 400 Bad Request; 403 Forbidden; 404 Not Found
PUT	/api/categories/{id}	Updates an existing event category.	Organiser	{ categoryName, distanceKm, entryFee, maximumParticipants }	200 OK – updated; 400 Bad Request; 403 Forbidden; 404 Not Found
DELETE	/api/categories/{id}	Deletes an event category.	Organiser	None	204 No Content – deleted; 403 Forbidden; 404 Not Found

GET	/api/events/{eventId}/enrolments	Returns all participant enrolments for an organiser's event.	Organiser	None	200 OK – enrolments returned; 403 Forbidden; 404 Not Found
GET	/api/enrolments/{id}	Returns details of an enrolment for an organiser's event.	Organiser	None	200 OK – enrolment returned; 403 Forbidden; 404 Not Found
POST	/api/enrolments/{enrolmentId}/result	Captures a participant's result for a completed event enrolment.	Organiser	{ finishTime, position, status }	201 Created – result recorded; 400 Bad Request; 403 Forbidden; 404 Not Found; 409 Conflict – result already exists
PUT	/api/results/{id}	Updates a participant's existing result.	Organiser	{ finishTime, position, status }	200 OK – result updated; 400 Bad Request; 403 Forbidden; 404 Not Found
GET	/api/results/{id}	Returns a specific	Organiser	None	200 OK – result returned; 403

		participant result.			Forbidden; 404 Not Found
GET	/api/events/{eventId}/results	Returns all recorded results for a specific organiser's event.	Organiser	None	200 OK – results returned; 403 Forbidden; 404 Not Found
DELETE	/api/results/{id}	Removes an incorrectly captured participant result.	Organiser	None	204 No Content – result deleted; 403 Forbidden; 404 Not Found

HTTP Method:

The HTTP Method identifies the type of operation that the client requests from the API. RaceDay primarily uses GET to retrieve information, POST to create or submit information, PUT to update resources and DELETE to remove resources. These methods follow standard HTTP request-method semantics (Mozilla, 2025).

Route:

The Route specifies the API address used to access a particular resource or operation. For example, /api/events represents the RaceDay events resource, while /api/events/{id} identify a specific event. ASP.NET Core Web APIs use routing to map incoming HTTP requests to the appropriate controller actions (Microsoft, 2026a).

Description:

The Description explains the purpose of the endpoint and the operation it performs. This makes the API plan easier to understand and provides a clear reference when the endpoints are implemented during Part 2.

Role Required:

The Role Required identifies which RaceDay users are authorised to access an endpoint. RaceDay uses the **Organiser** and **Participant** roles. Role-based authorisation allows access to particular application resources and operations to be restricted according to the authenticated user's assigned role (Microsoft, 2026b).

Request Body:

The Request Body identifies the information that the client must send to the API when creating or updating data. For example, registering an account requires information such as a name, email address, and password. ASP.NET Core Web APIs can receive structured request data and commonly exchange this information using JSON (Microsoft, 2026c).

Expected Response:

The Expected Response describes the HTTP status codes that the API may return after processing a request. For example, 200 OK represents a successful request, 201 Created indicates that a resource was successfully created, 400 Bad Request indicates an invalid request and 404 Not Found indicates that the requested resource could not be found. Including successful and unsuccessful responses makes the expected behaviour of each RaceDay endpoint clear before implementation (Fielding, Nottingham and Reschke, 2022; Microsoft, 2026a).

Section C: SQL Database Script

The SQL script creates the RaceDay database and builds all the tables designed in the ERD. It uses primary keys, foreign keys, and basic constraints to keep the data organised and maintain relationships between organisers, participants, events, enrolments, and results (Microsoft, 2026).

Database Creation

The script begins by creating the RaceDayDB database and selecting it as the active database. This provides a dedicated environment for storing all RaceDay data.

UserRole Table

The UserRole table stores the two system roles: Organiser and Participant. Each role has a unique RoleId, which is later referenced by the Organizers and Participants tables to identify the user's role.

Organizers Table

The Organizers table stores organiser account information, including their name, email, password, and phone number. OrganiserId is the primary key, while RoleId is a foreign key linking each organiser to the UserRole table.

Participants Table

The Participants table stores participant account and profile details. It includes personal information such as the participant's name, email address, phone number, and date of birth. Like organisers, participants are linked to UserRole through RoleId.

Events Table

The Events table stores all running, walking, and cycling events created by organisers. Each event belongs to one organiser through the OrganiserId foreign key, allowing the system to identify who manages each event.

EventCategories Table

The EventCategories table stores the different race categories offered within an event, such as 5 km, 10 km, or 21.1 km. This prevents event information from being duplicated and allows one event to have multiple categories.

Enrolments Table

The Enrolments table records participants entering event categories. It connects Participants and EventCategories using foreign keys and includes the participant's race number and enrolment status. A unique constraint prevents participants from registering for the same category more than once.

Results Table

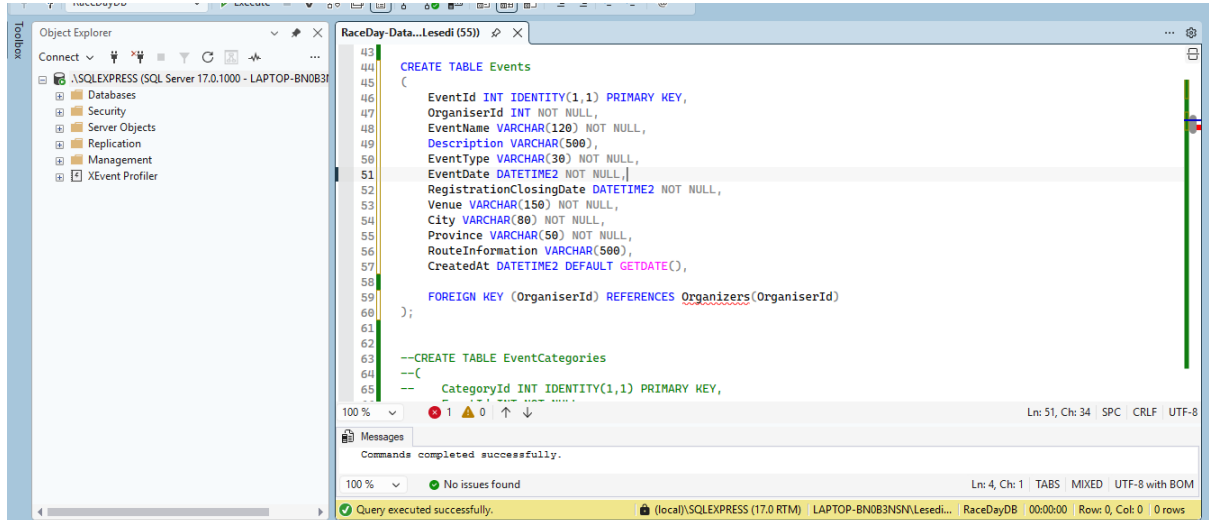
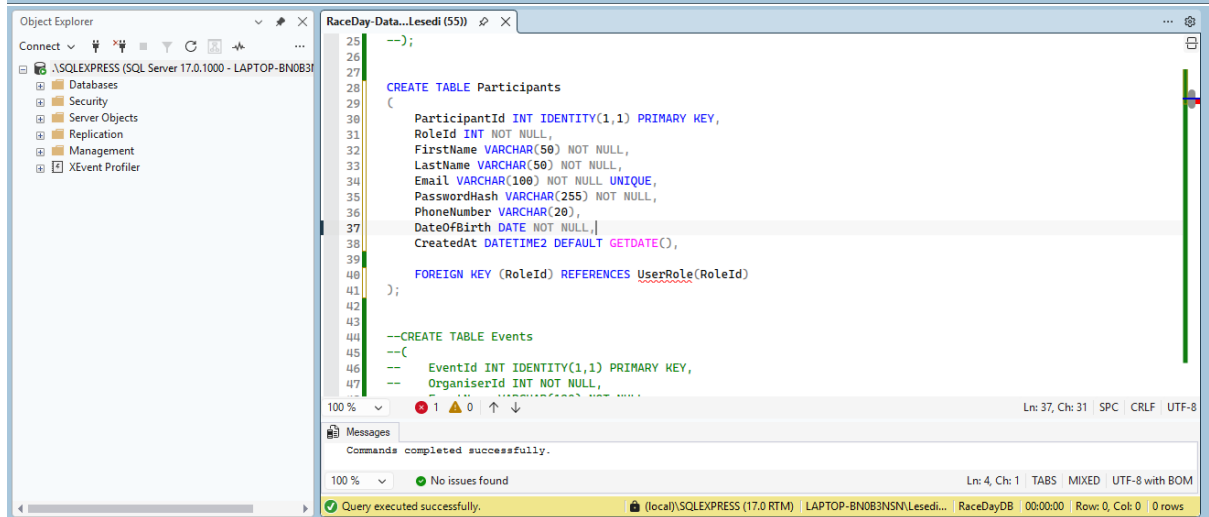
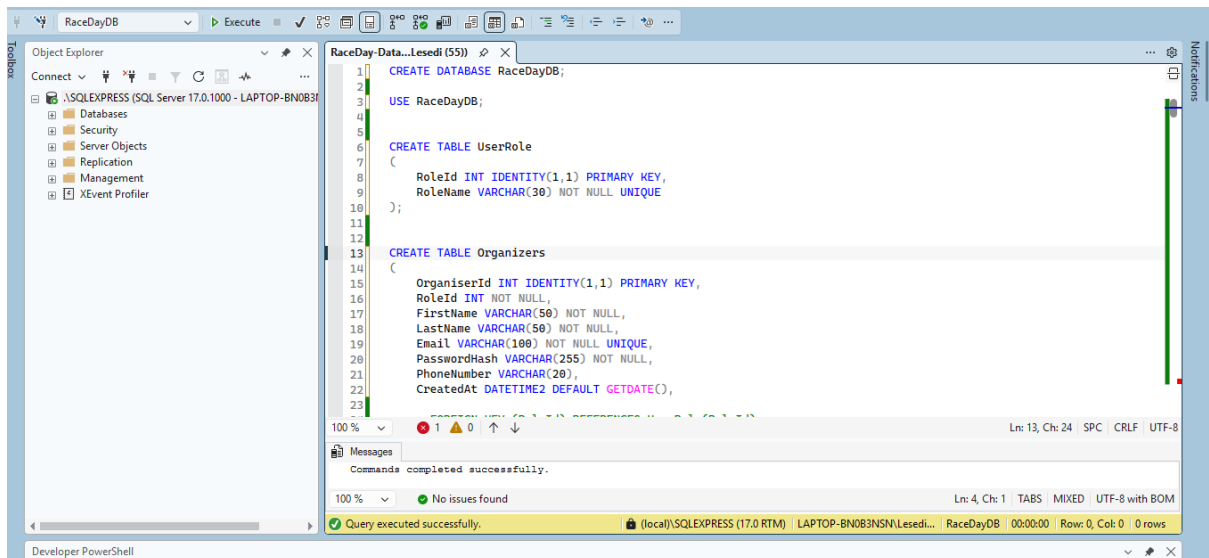
The Results table stores participant performance after an event. Each result belongs to one enrolment and records the participant's finishing time, position, and race status. This allows RaceDay to maintain each participant's personal race history.

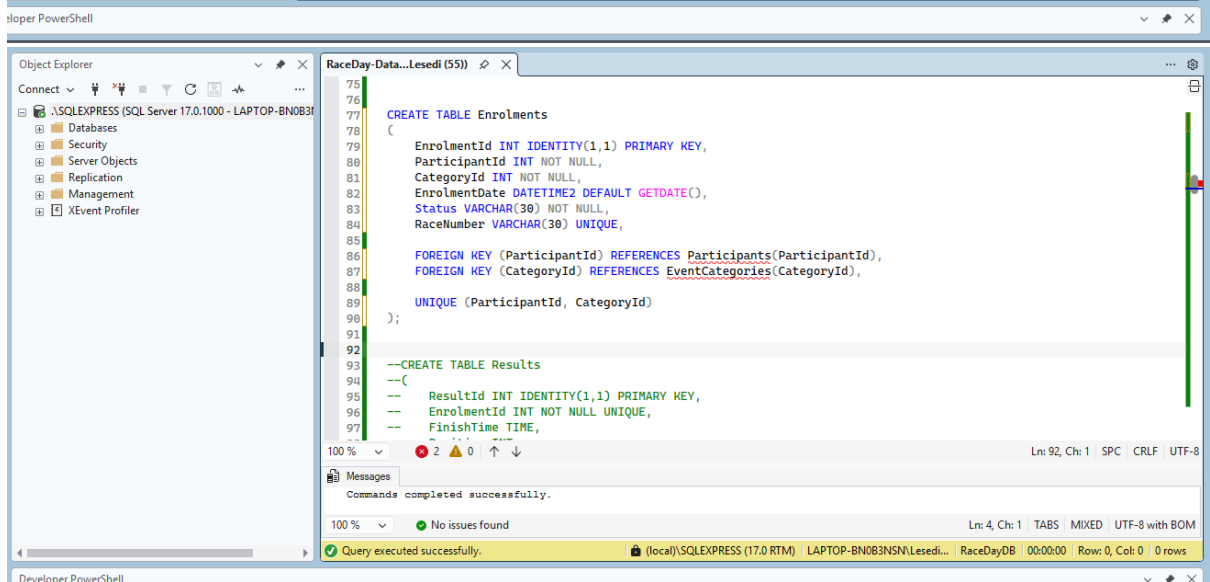
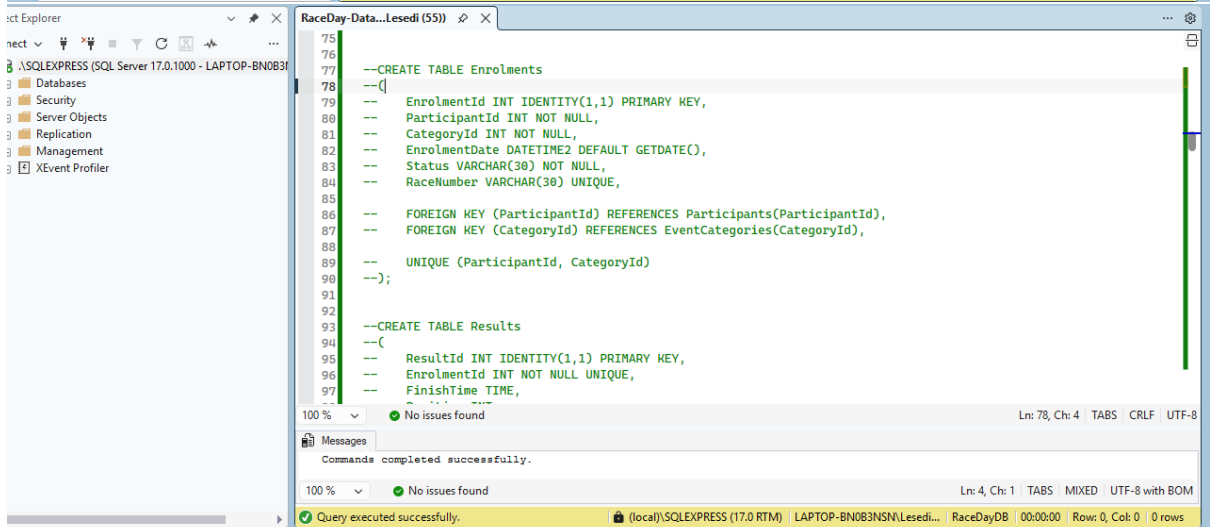
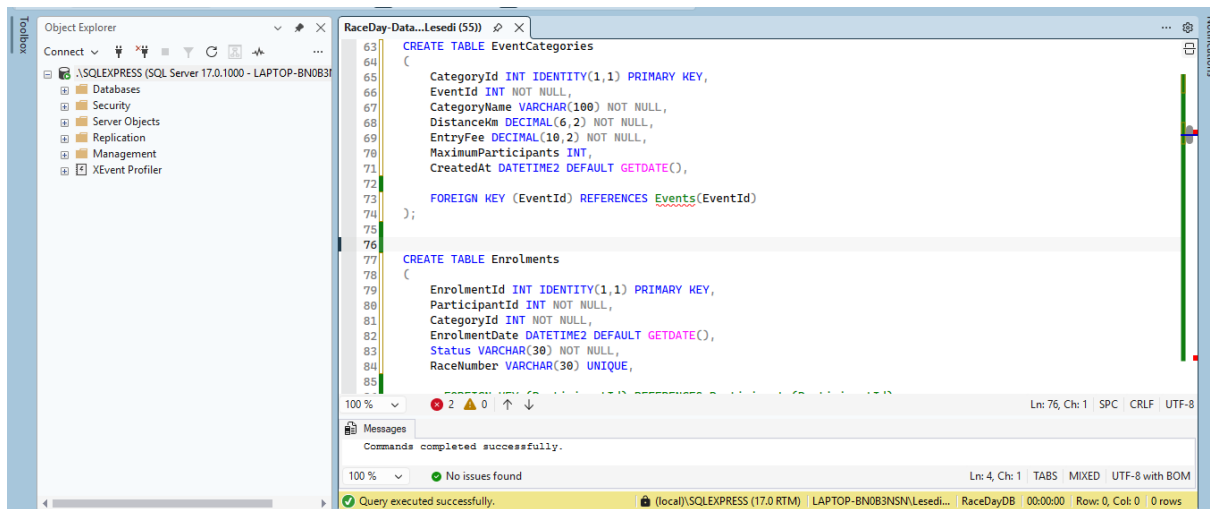
Sample Data

The script inserts sample records for two organisers, two participants, three events, event categories, enrolments, and results. The sample data allows the database to be tested immediately after it is created.

Testing Queries

The final section of the script uses SELECT statements and INNER JOIN queries to display organisers, participants, events, enrolments, and race results. These queries verify that the relationships between the tables were created successfully.





Object Explorer

Connect

SQLSERVER (SQL Server 17.0.1000 - LAPTOP-BN0B30)

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

RaceDay-Data...Lesedi (55)

```
CREATE TABLE Results
(
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentId INT NOT NULL UNIQUE,
    FinishTime TIME,
    Position INT,
    Status VARCHAR(50) NOT NULL,
    RecordedAt DATETIME2 DEFAULT GETDATE(),
    FOREIGN KEY (EnrolmentId) REFERENCES Enrolments(EnrolmentId)
);

-- Insert user roles
INSERT INTO UserRole (RoleName)
VALUES
    ('Organiser'),
    ('Participant');

-- Insert organisers
--INSERT INTO Organisers
--(
--    RoleId,
--    FirstName,
--    LastName,
--    Email,
--    PasswordHash,
--    PhoneNumber
--)
--VALUES
--(
--    1,
--    'Thabo',
--    'Mokone',
--    'thabo@raceday.co.za',
--    'password123',
--    '0821111111'
--),
--(
--    1,
--    'Maledi',
--    'Mhahlo',
--    'maledi@raceday.co.za',
--    'password456',
--    '0822222222'
--);

-- Insert participants
--INSERT INTO Participants
--(
--    RoleId,
--    FirstName,
--    LastName,
--    Email,
--    PasswordHash,
--    PhoneNumber,
--    DateOfBirth
--)
--VALUES
--(
--    2,
--    'Sipho',
--    'Dlamini',
--    'sipho@email.com',
--    'password123',
--    '0831111111',
--    '1998-05-14'
--),
--(
--    2,
--    'Lerato',
--    'Mkosi',
--    'lerato@email.com',
--    'password456',
--    '0832222222',
--    '2001-09-20'
--);
```

68 %

Ln: 114, Ch: 23 SPC CRLF UTF-8

Messages

Commands completed successfully.

Completion time: 2024-09-03T12:30:05.4976489+02:00

68 %

No issues found

Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully.

(local)\SQLSERVER (17.0 RTM) LAPTOP-BN0B30NSN\Lesedi... RaceDayDB | 00:00:00 | Row: 0, Col: 0 | 0 rows

Object Explorer

Connect

SQLSERVER (SQL Server 17.0.1000 - LAPTOP-BN0B30)

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

RaceDay-Data...Lesedi (55)

```
Insert participants
INSERT INTO Participants
(
    RoleId,
    FirstName,
    LastName,
    Email,
    PasswordHash,
    PhoneNumber,
    DateOfBirth
)
VALUES
(
    2,
    'Sipho',
    'Dlamini',
    'sipho@email.com',
    'password123',
    '0831111111',
    '1998-05-14'
),
(
    2,
    'Lerato',
    'Mkosi',
    'lerato@email.com',
    'password456',
    '0832222222',
    '2001-09-20'
);
```

68 %

Ln: 130, Ch: 27 SPC CRLF UTF-8

Messages

Commands completed successfully.

Completion time: 2024-09-03T12:30:05.4976489+02:00

68 %

No issues found

Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully.

(local)\SQLSERVER (17.0 RTM) LAPTOP-BN0B30NSN\Lesedi... RaceDayDB | 00:00:00 | Row: 0, Col: 0 | 0 rows

Object Explorer

Connect

SQLSERVER (SQL Server 17.0.1000 - LAPTOP-BN0B30)

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

RaceDay-Data...Lesedi (55)

```
Insert participants
INSERT INTO Participants
(
    RoleId,
    FirstName,
    LastName,
    Email,
    PasswordHash,
    PhoneNumber,
    DateOfBirth
)
VALUES
(
    2,
    'Sipho',
    'Dlamini',
    'sipho@email.com',
    'password123',
    '0831111111',
    '1998-05-14'
),
(
    2,
    'Lerato',
    'Mkosi',
    'lerato@email.com',
    'password456',
    '0832222222',
    '2001-09-20'
);
```

68 %

Ln: 158, Ch: 7 SPC CRLF UTF-8

Messages

Commands completed successfully.

Completion time: 2024-09-03T12:30:05.4976489+02:00

68 %

No issues found

Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM

Developer PowerShell

Object Explorer

Connect v [Icons]

SQL Server 17.0.1000 - LAPTOP-BN0B3...

- Databases
- Security
- Server Objects
- Replication
- Management
- XEvent Profiler

RaceDay-Data...Lesedi (551)

```
--(1, 4, 'Confirmed', 'COM001');
--(2, 6, 'Confirmed', 'GCC0001');

---- Insert results
INSERT INTO Results
(
  EnrolmentId,
  FinishTime,
  Position,
  Status
)
VALUES
(
  1,
  '00:52:14',
  35,
  'Finished'
),
(
  2,
  '01:58:40',
  21,
  'Finished'
);

-- Display all tables
SELECT * FROM UserRole;
SELECT * FROM Organizers;
```

68 % [Icons] Ln: 278, Ch: 7 SPC CRLF UTF-8

Messages

Commands completed successfully.

Completion time: 2024-09-12T12:30:05.4976489+02:00

68 % [Icons] No issues found Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully. (local)\SQLEXPRESS (17.0 RTM) | LAPTOP-BN0B3NSN\Lesedi... RaceDayDB | 00:00:00 | Row: 0, Col: 0 | 0 rows

Object Explorer

Connect v [Icons]

SQL Server 17.0.1000 - LAPTOP-BN0B3...

- Databases
- Security
- Server Objects
- Replication
- Management
- XEvent Profiler

RaceDay-Data...Lesedi (551)

```
---- Display participant enrolments
SELECT
  Participants.FirstName,
  Participants.LastName,
  Events.EventName,
  EventCategories.CategoryName,
  Enrolments.RaceNumber,
  Enrolments.Status
FROM Enrolments
INNER JOIN Participants
  ON Enrolments.ParticipantId = Participants.ParticipantId
INNER JOIN EventCategories
  ON Enrolments.CategoryId = EventCategories.CategoryId
INNER JOIN Events
  ON EventCategories.EventId = Events.EventId;

---- Display participant results
SELECT
  Participants.FirstName,
  Participants.LastName,
  Events.EventName,
  EventCategories.CategoryName,
  Results.FinishTime,
  Results.Position,
  Results.Status
FROM Results
INNER JOIN Enrolments
  ON Results.EnrolmentId = Enrolments.EnrolmentId
INNER JOIN Participants
  ON Enrolments.ParticipantId = Participants.ParticipantId
INNER JOIN EventCategories
```

68 % [Icons] Ln: 327, Ch: 5 SPC CRLF UTF-8

Messages

Commands completed successfully.

Completion time: 2024-09-12T12:30:05.4976489+02:00

68 % [Icons] No issues found Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully. (local)\SQLEXPRESS (17.0 RTM) | LAPTOP-BN0B3NSN\Lesedi... RaceDayDB | 00:00:00 | Row: 0, Col: 0 | 0 rows

Referencing:

Section A: Entity Relationship Diagram (ERD)

IBM, 2025. *What is database normalization?* IBM. Available at: [IBM – Database Normalization](#) (Accessed: 18 August 2026).

Microsoft, 2026. *Primary and foreign key constraints – SQL Server*. Microsoft Learn. Available at: [Microsoft Learn – Primary and Foreign Key Constraints](#) (Accessed: 18 August 2026).

Section B: API Endpoint Plan

Fielding, R., Nottingham, M. and Reschke, J., 2022. *HTTP Semantics (RFC 9110)*. RFC Editor. Available at: [RFC 9110 – HTTP Semantics](#) (Accessed: 20 August 2026).

Microsoft, 2026a. *Controller action return types in ASP.NET Core Web API*. Microsoft Learn. Available at: [Microsoft Learn – Controller action return types](#) (Accessed: 20 August 2026).

Microsoft, 2026b. *Role-based authorization in ASP.NET Core*. Microsoft Learn. Available at: [Microsoft Learn – Role-based authorization](#) (Accessed: 20 August 2026).

Microsoft, 2026c. *Format response data in ASP.NET Core Web API*. Microsoft Learn. Available at: [Microsoft Learn – Format response data](#) (Accessed: 20 August 2026).

Mozilla, 2025. *HTTP request methods*. MDN Web Docs. Available at: [MDN – HTTP request methods](#) (Accessed: 20 August 2026).

Section C: SQL Database Script

Coronel, C. and Morris, S. (2023) *Database Systems: Design, Implementation, & Management*. 14th edn. Boston: Cengage Learning.

Elmasri, R. and Navathe, S.B. (2021) *Fundamentals of Database Systems*. 7th edn. Harlow: Pearson.

Microsoft (2026) *CREATE TABLE (Transact-SQL)*. Microsoft Learn. Available at: <https://learn.microsoft.com/sql/t-sql/statements/create-table-transact-sql> (Accessed: 3 September 2026).

Microsoft (2026) Primary and Foreign Key Constraints. Microsoft Learn. Available at: <https://learn.microsoft.com/sql/relational-databases/tables/primary-and-foreign-key-constraints> (Accessed: 3 September 2026).